

FIAP — SOAT SOFTWARE ARCHITECTURE PÓS-TECH

CatCar Platform

Tech Challenge — Fase 3
Documento Consolidado de Entrega

Grupo: Grupo 274

Integrante: Davi Holanda

Turma / Curso: SOAT — Software Architecture Pós-Tech FIAP

Data: 18 de setembro de 2026

1. Identificação do grupo

Campo	Informação
Nome do grupo	Grupo 274
Projeto	CatCar Platform — Plataforma Integrada de Gestão de Oficina Mecânica
Integrante	Davi Holanda
Turma / Curso	SOAT — Software Architecture Pós-Tech FIAP

2. Repositórios entregues

A solução é composta por um meta-repositório e quatro repositórios independentes, todos na organização `Rise0n-CatCar` :

Repositório	Responsabilidade
<code>catcar-platform</code>	Meta-repositório, documentação consolidada, automação local e composição dos submódulos.
<code>catcar-app</code>	Monólito modular .NET 10, DDD, EF Core, Wolverine Outbox e Aspire AppHost.
<code>catcar-auth-function</code>	Azure Function serverless para autenticação de cliente por CPF e emissão de JWT.
<code>catcar-database-infra</code>	Terraform para PostgreSQL Flexible Server, Key Vault, DNS privado e segredos.
<code>catcar-kubernetes-infra</code>	Terraform para VNet, AKS, ACR, APIM, Azure Monitor, alertas e workbooks.

3. Introdução: Organização do Repositório e Metodologia

Organização em Meta-Repositório (Umbrella Workspace)

A plataforma é organizada como um *Umbrella Workspace*: o meta-repositório `catcar-platform` coordena quatro submódulos segregados e autônomos — `catcar-app`, `catcar-auth-function`, `catcar-database-infra` e `catcar-kubernetes-infra`. Cada submódulo mantém seu próprio ciclo de vida, histórico e pipeline de CI/CD, com responsabilidades claramente delimitadas entre a aplicação, a autenticação serverless e as duas camadas de infraestrutura.

Essa topologia reduz o acoplamento entre entregas, permite evolução e publicação independentes e limita o raio de impacto (*blast radius*) de alterações e incidentes. Ao mesmo tempo, a experiência de desenvolvimento permanece unificada: o `catcar-platform` concentra a documentação, o Makefile e a orquestração local por .NET Aspire, coordenando os componentes sem eliminar sua autonomia operacional.

Metodologia e Papel da Inteligência Artificial

A Inteligência Artificial foi empregada como multiplicação de mão de obra: um recurso para ampliar, acelerar e sistematizar a execução do trabalho de engenharia. Todos os conceitos arquiteturais, padrões de design e decisões técnicas foram originados diretamente do conhecimento arquitetural e da experiência de engenharia do autor, **Davi Holanda**.

A assistência de IA potencializou a materialização dessas decisões, sem substituir sua autoria técnica. Entre os elementos definidos pelo autor estão DDD, Vertical Slices, Transactional Outbox, autenticação serverless, topologia de nuvem, isolamento de rede Zero-Trust e credenciais federadas OIDC.

4. Documentação e evidências técnicas

Arquitetura, DDD e decisões

- [Diagrama de componentes e topologia em nuvem](#)
- [Diagramas de sequência — autenticação e ordem de serviço](#)
- [Modelo Entidade-Relacionamento \(ER\)](#)
- [Artefatos DDD: visão geral](#)
- [Context Map, Event Storming, estrutura de módulos e glossário](#)

RFCs e ADRs

- [RFC 001 — seleção do provedor de nuvem \(Microsoft Azure\)](#)
- [RFC 002 — banco gerenciado PostgreSQL 17](#)
- [RFC 003 — autenticação serverless com Azure Functions](#)
- [ADR 001 — comunicação assíncrona com Transactional Outbox](#)
- [ADR 002 — HPA e rate limiting no APIM](#)
- [ADR 003 — topologia de repositórios e composição Aspire](#)

Operação, segurança e APIs

- [Runbook de bootstrap Azure](#)
- [Relatório de vulnerabilidades](#)
- [Relatório de auditoria e conformidade da Fase 3](#)
- [Swagger UI](#) — `/swagger`
- [Scalar API Reference](#) — `/docs`
- [OpenAPI v3 JSON](#) — `/openapi/v1.json`
- [Collection Postman v2.1.0](#)
- [Environment template Postman](#)

5. Acesso do avaliador `soat-architecture`

Confirmação registrada: o controle de requisitos do projeto registra `soat-architecture` como colaborador com permissão de *push* nos quatro repositórios de entrega. A evidência rastreável está em [catcar-app/docs/status-requisitos-entregaveis.md](#), item F3-ENT-06.

Repositório	Acesso a confirmar no GitHub	Evidência a anexar ao envio
catcar-app	Colaborador <code>soat-architecture</code> com acesso ativo.	Captura de tela de Settings → Collaborators and teams ou aceite de convite.
catcar-auth-function	Colaborador <code>soat-architecture</code> com acesso ativo.	Captura de tela de Settings → Collaborators and teams ou aceite de convite.
catcar-database-infra	Colaborador <code>soat-architecture</code> com acesso ativo.	Captura de tela de Settings → Collaborators and teams ou aceite de convite.
catcar-kubernetes-infra	Colaborador <code>soat-architecture</code> com acesso ativo.	Captura de tela de Settings → Collaborators and teams ou aceite de convite.

Procedimento de comprovação antes da submissão

1. Um administrador da organização abre cada repositório e acessa **Settings** → **Collaborators and teams**.
2. Confirma que `soat-architecture` aparece como colaborador ativo — não apenas com convite pendente — com permissão mínima de leitura (o registro interno indica *push*).
3. Registra uma captura de tela por repositório, preservando o nome do repositório, o usuário e o status de acesso; não expõe tokens ou outros dados sensíveis.
4. Anexa as quatro evidências ao PDF final ou ao campo de observações do Portal do Aluno e mantém os repositórios acessíveis para a correção.

6. Resumo executivo de aderência aos requisitos obrigatórios

Requisito obrigatório	Evidência de implementação
API Gateway + autenticação serverless	Azure API Management aplica política JWT e roteia <code>POST /api/auth/customer</code> para a Azure Function. A Function valida CPF, consulta cliente ativo e emite JWT HMAC-SHA256.
Banco de dados gerenciado	Azure Database for PostgreSQL Flexible Server 17 com subnet delegada, DNS privado, SSL e segredos protegidos no Azure Key Vault.
Kubernetes escalável	AKS multi-AZ com Azure CNI, Workload Identity e HPA baseado em CPU/memória, configurado de 2 a 10 réplicas.
Infraestrutura como código	Os stacks <code>catcar-database-infra</code> e <code>catcar-kubernetes-infra</code> descrevem a infraestrutura Azure obrigatória integralmente em Terraform, com pipelines próprias de validação e aplicação.
Observabilidade e workbooks	OpenTelemetry, Application Insights, Log Analytics, health checks, alertas de latência/CPU/memória/uptime e workbook operacional permitem acompanhamento técnico e de negócio.
CI/CD dual-environment e governança	Pipelines independentes publicam Homologação a partir de <code>develop</code> e Produção a partir de <code>main</code> ; o bootstrap configura OIDC, ambientes e proteção de branches com PR, revisão e bloqueio de force-push/exclusão.